

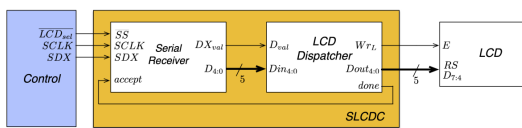
## 1 SLCDC - Serial LCD Controller

O módulo *Serial LCD Controller* (SLCDC) implementa a interface entre o sistema de controlo (*Control*, em software) e o LCD. O seu objetivo é receber dados em série, enviados pelo *Control* via protocolo dedicado, e entregá-los paralelamente ao LCD.

A comunicação entre o *Control* e o SLCDC é feita utilizando apenas três sinais: **notSS**, **SCLK** e **SDX**, minimizando o número de interligações físicas.

O SLCDC é composto por dois blocos principais, tal como representado na Figura 1:

- **Serial Receiver** — responsável pela receção, reconstrução e validação dos dados transmitidos 3. Dada a sua natureza genérica, este bloco foi também utilizado na construção do SRC, com as devidas adaptações;
- **LCD Dispatcher** — responsável pela entrega dos dados válidos ao LCD. Dada a sua natureza genérica, este bloco foi também utilizado na construção do SRC, com as devidas adaptações;



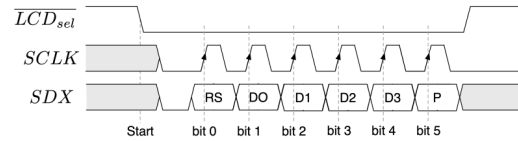
**Figura 1:** Diagrama de blocos do SLCDC.

### 1.1 Protocolo de Comunicação

A trama de dados enviada pelo *Control*, segundo o protocolo representado na Figura 2, para o SLCDC é composta por:

- 1 bit de controlo (**RS**) — indica se se trata de um comando (RS=0) ou de dados (RS=1);
- 4 bits de dados — representam o nibble a ser escrito;
- 1 bit de paridade — paridade ímpar calculada sobre os cinco bits anteriores.

A receção é iniciada com uma transição descendente do sinal **notSS** (início da transmissão). A partir daí, cada bit é lido nas transições ascendentes do sinal **SCLK**.



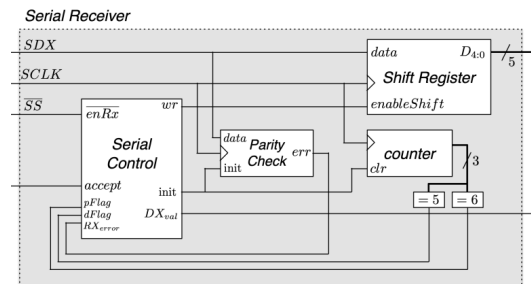
**Figura 2:** Protocolo de comunicação com o SLCDC.

### 1.2 Serial Receiver

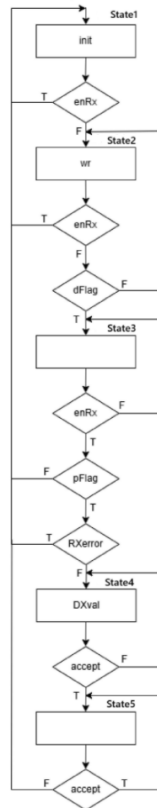
O bloco *Serial Receiver* é responsável por reconstruir os dados recebidos bit a bit e validar a paridade. É composto por quatro sub-blocos:

- **Serial Control** — máquina de estados que gere a receção (ver Figura 4);
- **Shift Register** — armazena os bits recebidos;
- **Counter** — conta os bits recebidos;
- **Parity Check** — verifica se a paridade da trama é ímpar.

Após a receção correta de uma trama (6 bits), o sinal **Dval** é ativado, sinalizando ao *LCD Dispatcher* que uma nova trama está pronta a ser processada. O valor da trama é disponibilizado no barramento **Dout[4:0]**.



**Figura 3:** Bloco Serial Receiver do SLCDC.



**Figura 4:** Máquina de estados do Serial Receiver.

### 1.3 LCD Dispatcher

O *LCD Dispatcher* é responsável por processar as tramas válidas recebidas do *Serial Receiver* e encaminhá-las para o LCD. Este módulo monitoriza o sinal **Dval**, e quando este se encontra ativo, envia o conteúdo presente no barramento **Dout[4:0]**:

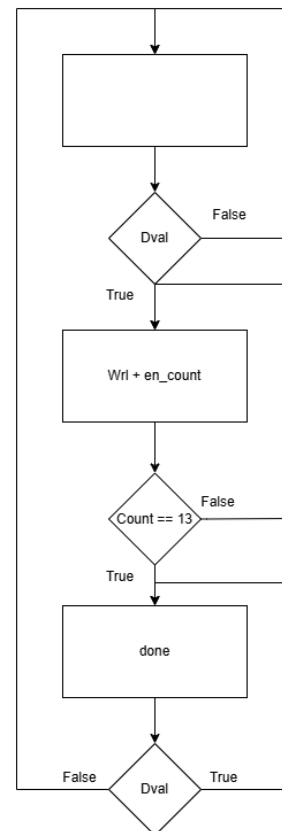
- Se **RS = 0**, trata-se de um comando (por exemplo, limpar o ecrã ou posicionar o cursor);
- Se **RS = 1**, trata-se de um dado (por exemplo, um carácter a escrever no LCD).

Após interpretar a trama, o *LCD Dispatcher* ativa o sinal **WrL** para transferir o nibble correspondente para o LCD. Nesta fase é também ligado um sinal **en\_count**, que liga um contador interno do bloco, que serve para garantir que houve tempo suficiente para ocorrer a escrita no lcd. Quando este contador chega a 13, podemos passar para

o próximo estado, garantindo que houve o tempo necessário de escrita. No estado seguinte, o dispatcher emite o sinal **done** para informar o *Serial Receiver* de que a trama foi processada, permitindo assim que o canal de receção série seja libertado de forma imediata.

Importa salientar que, mesmo que o LCD ainda esteja a executar internamente a instrução recebida, o canal de receção série não precisa de aguardar por essa execução. Esta abordagem permite que o *Control* continue a enviar novas tramas sem interrupções, assegurando uma comunicação eficiente e contínua entre o sistema de controlo e o LCD.

Este comportamento corresponde à máquina de estados representada na Figura 5.



**Figura 5:** Máquina de estados do LCD Dispatcher.

## 2 Control - Software

A componente de *Control*, desenvolvida em Kotlin, implementa a lógica de alto nível do sistema de jogo da roleta, interagindo com os periféricos (teclado, LCD, display da roleta) através de módulos especializados. Esta arquitetura modular e bem separada permite manter o código coeso, testável e facilmente extensível.

### 2.1 SerialEmitter

O **SerialEmitter** é o módulo responsável por enviar tramas codificadas através da interface série, usadas para comunicar com os módulos SLCDC (LCD) e SRC (roleta).

Cada trama contém:

- Bits de dados (4 ou 8 bits conforme destino);
- Bit RS ou comando (LCD ou Roleta);
- Bit de paridade ímpar.

As funções principais incluem:

- **send(destination, data, size)** — envia a trama codificada com 1 bit de paridade ímpar;
- **init()** — inicializa ambos os destinos e limpa a linha de clk;
- **pulseClock()** - impõe um ciclo de SCLK clock;

### 2.2 LCD

Este módulo fornece funções de alto nível para interação com o display LCD. Usa o **SerialEmitter** (modo série) ou interface paralela para enviar comandos ou dados ao LCD.

Funções principais:

- **init()** — Sequência de inicialização do LCD;
- **write(Char), write(String)**
- **writeNibbleSerial, writeNibbleParallel;**
- **cursor(line, column), clear().**

As instruções são convertidas internamente para tramas apropriadas, com distinção entre comandos (RS=0) e dados (RS=1).

### 2.3 TUI - Text User Interface

O objetivo do TUI é permitir abstrair a interação com o utilizador final, através do LCD e teclado. Para isso ele deve fornecer utilitários para apresentar menus, pedir inputs, mostrar mensagens, limpar o ecrã, entre outros.

Este módulo serve de ponte entre a lógica do jogo e os periféricos visuais e de input.

Para isso foram implementadas as funções:

- **init()** — Inicialização do LCD e do KBD;
- **writeMessage()** - Escrita de uma string
- **getKey()** - Guarda o valor da key premida;
- **isMaintenance()** - Verifica se o sistema está no modo de manutenção.